THADOMAL SHAHANI ENGINEERING COLLEGE

Department Of Artificial Intelligence & data Science

EXPERIMENT-01

Name:- Gyaan Achantani **Date:-** 14th July,26

Div:- T-1 **Batch:-** T-11 **R.no:-** 2401001

Subject:- Agile Software Development & DevOPS Lab

LO- 02

LO – Use Git Effectively for version control, source code Management and collaborative team development

AIM:- To Perform Version Control Operations Using Git.

Theory:- Git is a distributed version control system designed to track changes in source code during software development. It functions as a comprehensive, historical record of a project, allowing multiple contributors to work simultaneously on the same codebase without overwriting each other's work. By maintaining a local copy of the entire project history on every developer's machine, Git provides high performance, data integrity, and the ability to branch, experiment, and revert changes efficiently, ensuring that no progress is ever permanently lost.

The Three Git Zones

- **Working Directory:** Your local folder where you actively edit files.

- **Staging Area (Index):** A buffer where you "prepare" changes. You selectively add files here to decide what will be included in the next "snapshot."
- **Repository (HEAD):** The database where Git stores permanent snapshots (commits).

Theory of Git Operations:-

 init / clone: Establishes the .git directory, which acts as the database for all version history.

 config: Establishes user identity metadata, which is attached to every commit so the system knows who authorized a change.

 add: Updates the index. It does not save the file to the repository; it simply marks it to be included in the next commit.

 commit: Creates a permanent, immutable snapshot of the staging area. Each commit is identified by a unique SHA-1 hash (the fingerprint), allowing Git to verify the integrity of the data.

 status: Queries the working directory against the index to identify "untracked," "modified," or "staged" files.

 push: Transfers local commit objects to a remote server (like GitHub) and updates the remote branch references.

 pull / fetch: Synchronizes your local repository with the remote. fetch only downloads the data; pull performs a fetch followed by a merge to bring your local files up to date.

branch: Creates a new pointer at the current commit.

 log / diff: Traverses the commit graph to display history or perform byte-by-byte comparisons between file states.

 stash: Takes current "dirty" (uncommitted) changes and stores them on a temporary stack, allowing you to clean the working directory to switch branches.

 revert / reset:

- revert creates a *new* commit that applies the inverse of an old one (maintaining history integrity).
- reset moves the branch pointer backward (rewriting history).
- • **`remote prune` (The "Housekeeper"):** If a colleague deletes a branch on GitHub, your computer still thinks it exists. Pruning deletes the references to remote branches that no longer exist on the server.
- • **`fetch --all` (The "Synchronization Broadcaster"):** Fetches updates from *every* remote repository configured, not just the one you are currently tracking.
- • **`push --force-with-lease` (The "Safe Overwriter"):** Unlike a standard `--force`, this checks to see if the remote branch has received new updates from others before allowing you to overwrite it. It prevents you from accidentally destroying someone else's work.

. show-branch (The "Ancestry Map"): This provides a compact view of commit relationships across different branches.

OUTPUT:-

GIT CLONE

```
PS C:\Users\Gyaan> git clone https://github.com/ACHANTANI-GYAAN/ASD-DO.git
Cloning into 'ASD-DO'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

GIT COMMIT

1.
```
PS C:\Users\Gyaan\ASD-DO> git commit -m "Initial Commit of index and style files"
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.
```

GIT STATUS

```
PS C:\Users\Gyaan\ASD-DO> git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html

nothing added to commit but untracked files present (use "git add" to track)
```

GIT LOG

```
PS C:\Users\Gyaan\ASD-DO> git log
commit 450ea4065277f456cb9ca90d16e8158bfdbeb280 (HEAD -> main, origin/main, origin/HEAD)
Author: ACHANTANI-GYAAN <gyaanachantani7@gmail.com>
Date:   Tue Jul 14 19:15:52 2026 +0530

    Initial commit
```

GIT PUSH

```
PS C:\Users\Gyaan\ASD-DO> git push origin main
info: please complete authentication in your browser...
Everything up-to-date
```

GIT CHECKOUT

```
PS C:\Users\Gyaan\ASD-DO> git checkout feature-login
A       index.html
A       style.css
Switched to branch 'feature-login'
```

GIT BRANCH FEATURE LOGIN

```
PS C:\Users\Gyaan\ASD-DO> git branch -d feature-login
Deleted branch feature-login (was 450ea40).
```

GIT DIFF

```
PS C:\Users\Gyaan\ASD-DO> git diff
diff --git a/index.html b/index.html
index b89cc6c..ccb6faf 100644
Binary files a/index.html and b/index.html differ
```

GIT BRANCH

```
PS C:\Users\Gyaan\ASD-DO> git branch -a
* main
  remotes/origin/HEAD -> origin/main
  remotes/origin/main
```

GIT STASH

```
PS C:\Users\Gyaan\ASD-DO> git stash
Saved working directory and index state WIP on main: 450ea40 Initial commit
```

GIT LOG

```
PS C:\Users\Gyaan\ASD-DO> git log --oneline
450ea40 (HEAD -> main, origin/main, origin/HEAD) Initial commit
```

GIT STASH LIST

```
PS C:\Users\Gyaan\ASD-DO> git stash list
stash@{0}: WIP on main: 450ea40 Initial commit
stash@{1}: WIP on main: 450ea40 Initial commit
```

GIT REMOTE

```
C:\Users\Gyaan\ASD-DO>git remote -v
origin  https://github.com/ACHANTANI-GYAAN/ASD-DO.git (fetch)
origin  https://github.com/ACHANTANI-GYAAN/ASD-DO.git (push)
upstream        https://github.com/ACHANTANI-GYAAN/ASD-01.git (fetch)
upstream        https://github.com/ACHANTANI-GYAAN/ASD-01.git (push)
```

GIT REVERT

```
Revert "Initial commit"

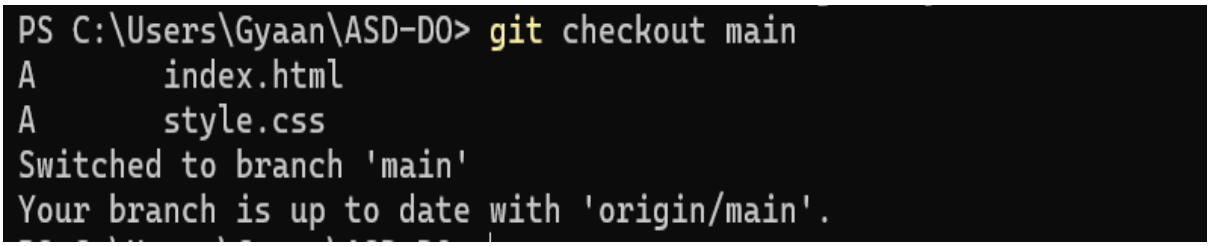
This reverts commit 450ea4065277f456cb9ca90d16e8158bfdbeb280.

# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch main
# Your branch is up to date with 'origin/main'.
#
# Changes to be committed:
#       deleted:    README.md
#
.git/COMMIT_EDITMSG [unix] (20:02 14/07/2026)                                 1,1 All
"~/ASD-DO/.git/COMMIT_EDITMSG" [unix] 13L, 345B
```

GIT RESET

```
C:\Users\Gyaan\ASD-DO>git reset --hard 450ea40
HEAD is now at 450ea40 Initial commit
```

GIT CHECKOUT MAIN



CONCLUSION:-

The completion of these version control operations has provided a comprehensive understanding of Git's functionality in managing source code evolution. By successfully performing everything from basic snapshotting and branching to complex history surgery and integrity checks, the experiment has established a solid foundation in modern development workflows**.**